Federated Memory Protocol Draft v1.1

Sruly Rosenblat, AI Disclosures Project
October 13, 2026

Glossary

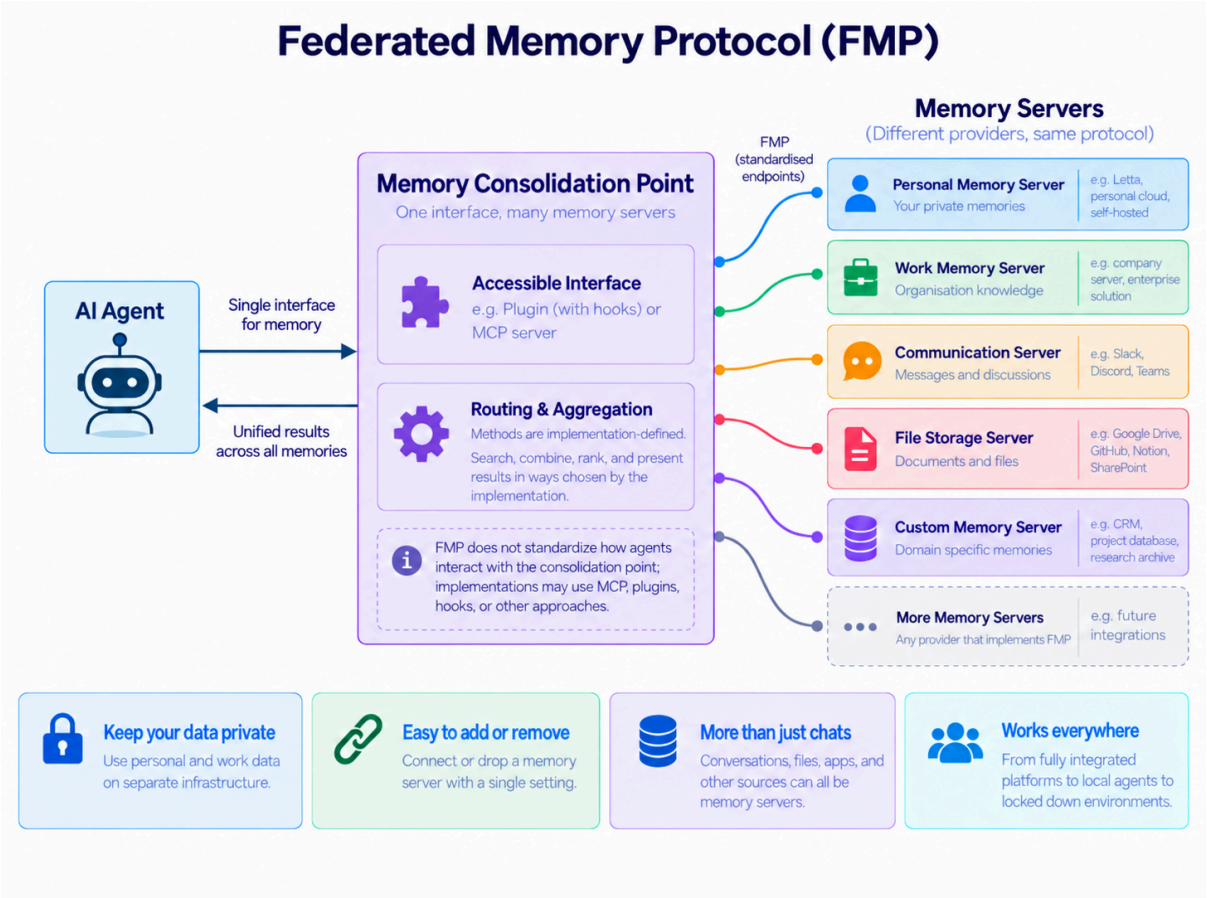
Memory - Data relevant to customising or refining the user's experience with an agent and aligning the agent more with the user. Memory can be context specific (such as a specific company or project). Memory can be shared across an organisation. Memory can be generated based on conversation data or it can be data from other apps such as discord or slack.

Memory Server - A server exposed by memory providers that allows clients (agents) to read and write memories stored in the providers memory platform. The protocol does not dictate how memories are stored, searched, retrieved or compiled; it just provides a standardised interface for agents to interact with memory.

Memory consolidation point - The consolidated interface through which the agent interacts with various memory systems. The interface can be an MCP server, it can also be a plugin. It can use hooks or it can just use tool calls. The protocol does not extensively enforce the memory consolidation point, only the memory server that enables it.

Overview

The Federated Memory Protocol (FMP) is a proposed protocol that will allow for users to connect one agent to multiple distinct memory stores without having them compete for space in the context window or installing separate MCP servers. This is done by standardising an interface for memory servers to expose that will allow for memories to be consolidated and exposed to the agent through one interface.



One FMP plugin can connect to multiple memory servers at once, allowing agents to utilize multiple data sources to accomplish their tasks. The federated approach also allows for more privacy as you dont need to duplicate data across the memory server.

The federated approach allows for a few distinct benefits:

1. The agent doesn't need to navigate multiple interfaces if memory is stored across different memory servers. It can search with one interface and can update memories uniformly.
2. The user has more control over where their memory is stored. And can specify that this memory can only be stored in their personal server or this memory should be used.
3. The memory consolidation point can be used as a gateway to clean up or gate memories across different systems.
4. The agent does not need to know what memory servers are in use or how memory is separated (although it can depend on implementation). It only needs to see how to interact with the memories it finds important.
5. Does not rely on a specific level of agent ability allows custom adapters to connect agents to memory.
6. So memory granularity has two levels A) different servers can refuse different memories and B) any one server can discriminate what memories the agent can upload.

Implementation details

The protocol is agnostic to how it's delivered. A *memory server* is any backend that exposes the FMP endpoints; it is not the same thing as an MCP server, which is just one way an agent can reach it.

There are three tiers of support, in order of preference:

1. **Native integration.** The harness speaks FMP directly. No plugin or MCP server needed.
2. **Plugin + bundled MCP server.** Where the harness supports plugins, the plugin gains access to hooks (e.g., to capture full transcripts automatically), and the bundled MCP server lets the model search and upload memories on demand.
3. **Plain MCP server.** Where neither of the above is available. The model can still search all memory servers and upload explicit memories, but won't have hooks, so full transcripts likely won't be captured.

In every tier, all connected memory servers expose the same endpoints, so the model can combine them into a single view of the user's history. The protocol says nothing about how a server captures, stores, or searches memory; it only standardizes the interface.

This means that different agents will have different ways of interacting with memory servers:

On supported platforms:

Adding a memory server will be as easy as adding a URL in the settings. The harness itself will deal with the complexity of MCP and Hook setup.

On local coding agents:

The agent will have full access to hooks, ability to upload full transcripts, and gather whatever context you can through a plugin. Plugins only need to be made once for agents by the community but a plugin importantly does not lock you down to one memory backend and one plugin can be used for different memory backends. A bundled MCP server will allow the agent to manually search or manually upload memories.

On more locked down platformers:

Can still search memories from all platforms, can still upload memories to all servers. But will likely not send full transcripts to the servers. MCP is sufficient to gain access to all memories and upload some.

The protocol does not assume a file system. It works on any agent to at least some extent.

Permissions

Each memory server can handle permissions in their own way, they can provide a URL for users to manually approve or reject memories. They can also allow memories to be written arbitrarily as long as the user is signed in (via OAuth or some other mechanism). Memories can be tagged to different users or different contexts

and those contexts can require different permissions within that memory server. Eg, context that will be shared across a company may require sign off from a company administrator. Agent editing can be disabled entirely and only be changed on a users dashboard.

So memory granularity has two levels 1) different servers can refuse different memories and 2) one server can discriminate what memories the agent can upload.

Memory Format

The memory format will defer to the OPMI semantic exchange format

Possible Endpoints

Method	Endpoint	Requirement	Purpose
GET	/fmp/info	Required	Discover capabilities and version
POST	/fmp/search	Optional	Search provider memories
POST	/fmp/read	Optional	Retrieve objects by ID
POST	/fmp/upload/inferences	Optional	Submit memory objects
POST	/fmp/upload/transcripts	Optional	Submit conversation history
POST	/fmp/upload/files	Optional	Submit external documents
GET	/fmp/read_inferences	Optional	Export memory history
GET	/fmp/read_transcripts	Optional	Export transcript history

`POST`	`/fmp/delete`	Optional	Request deletion of objects

Any of these can be disabled by the memory server while still being compliant with the proposed protocol.

The middle point between the federated memory servers and the agents, the memory consolidation point, decides how results are combined. It is not on the protocol to decide which memories are valuable, where memories are routed or where to upload memories too, just how memories can be federated. The plugin or MCP server decides how everything besides for data transfer is done.

Benefits of this approach

- Assumes nothing of either the producer or consumer. As long as MCP is supported, both consumption and explicit memories are supported. If hooks are also supported by a harness then every system connected gets a richer view of memory.
- If there is adoption the memory consolidation point can be integrated directly into the harness so a third party plugin or MCP server is not needed. If it is not integrated any harness can still be supported anyway via MCP and or hooks.
- Is not tied to an operating system. No need for a LLM to be able to read files.

Inspiration:
ARDS, HCP, Cognee, Letta trajectory, IBM memory protocol draft.